An Abstract Machine for a Simple Functional Language

1 INTRODUCTION

This is an abstract machine for a simple functional language in the AAA551 course at Korea University. It supports the following features:

- **number (integer) values** (`0`, `1`, `-1`, `2`, `-2`, `3`, `-3`, ...)
- **arithmetic operators**: addition (`+`) and multiplication (`*`)
- **immutable variable definitions** (`let`)
- **first-class functions** (`=>`)
- **recursive function definitions** (`def`)
- **lists** (e.g., `[1, 2, 3]`)
- **list construction** (`::`)
- **list pattern matching** (`match`)

This document is the specification of the language. First, Section 2 describes the concrete syntax, and Section 3 describes the abstract syntax with the definitional transition rules. Then, Section 4 describes the abstract machine for this language.

2 CONCRETE SYNTAX

The concrete syntax is written in a variant of the extended Backus–Naur form (EBNF). The notation `<nt>` denotes a nonterminal, and `"t"` denotes a terminal. We use `?` to denote an optional element and `+` (or `*`) to denote one or more (or zero or more) repetitions of the preceding element. We use `butnot` to denote a set difference to exclude some strings from a producible set of strings. We omit some obvious terminals using the ellipsis (...) notation.

```
// basic elements
<digit>    ::= "0" | "1" | "2" | ... | "9"
<number>   ::= "-"? <digit>+
<alphabet> ::= "A" | "B" | "C" | ... | "Z" | "a" | "b" | "c" | ... | "z"
<idstart>  ::= <alphabet> | "_"
<idcont>   ::= <alphabet> | "_" | <digit>
<keyword>  ::= "let" | "def" | "match" | "with"
<id>       ::= <idstart> <idcont>* butnot <keyword>

// expressions
<expr> ::= <number> | <expr> "+" <expr> | <expr> "*" <expr> | <id>
         | "(" <expr> ")" | "{" <expr> "}"
         | <id> "=>" <expr> | <expr> "(" <expr> ")"
         | "let" <id> "=" <expr> ";" <expr>
         | "def" <id> "(" <id> ")" "=" <expr> ";" <expr>
         | <expr> "::" <expr>
         | "[" <expr>? ("," <expr>)* "]"
         | "match" <expr> "with"
           "|" []              "=>" <expr>
           "|" <id> "::" <id> "=>" <expr>
```

The precedence and associativity of operators are defined as follows:

Operator	Associativity	Precedence
*	left	3
+	left	2
::	right	1

3 ABSTRACT SYNTAX

The abstract syntax of is defined as follows:

$$
\begin{array}{llll}
\text{Expressions} & \mathbb{E} \ni e ::= n & & \texttt{(Num)} \\
& \quad\quad\quad\quad\ \mid e + e & & \texttt{(Add)} \\
& \quad\quad\quad\quad\ \mid e * e & & \texttt{(Mul)} \\
& \quad\quad\quad\quad\ \mid x & & \texttt{(Id)} \\
& \quad\quad\quad\quad\ \mid \lambda x.e & & \texttt{(Fun)} \\
& \quad\quad\quad\quad\ \mid \texttt{def}\ x(x) = e;\ e & & \texttt{(Def)} \\
& \quad\quad\quad\quad\ \mid e(e) & & \texttt{(App)} \\
& \quad\quad\quad\quad\ \mid e :: e & & \texttt{(Cons)} \\
& \quad\quad\quad\quad\ \mid [e_1, \ldots, e_n] & & \texttt{(Lst)} \\
& \quad\quad\quad\quad\ \mid e \rhd (e \mid x_1, x_2, e) & & \texttt{(Mat)}
\end{array}
$$

where

$$
\begin{array}{lll}
\text{Numbers} & n \in \mathbb{Z} & \texttt{(BigInt)} \\
\text{Identifiers} & x \in \mathbb{X} & \texttt{(String)}
\end{array}
$$

The semantics of the remaining cases are defined with the following desugaring rules:

$$\mathcal{D}[\![\texttt{val}\ x = e_1;\ e_2]\!] = (\lambda x.\mathcal{D}[\![e_2]\!])(\mathcal{D}[\![e_1]\!])$$

The omitted cases recursively apply the desugaring rule to sub-expressions.

4 ABSTRACT MACHINE

We use the following abstract machine to define the semantics of the language:

$$
\begin{array}{llll}
\text{Values} & \mathbb{V} \ni v ::= n & & \texttt{(NumV)} \\
& \quad\quad\quad\ \mid \langle \lambda x.e, \rho \rangle & & \texttt{(CloV)} \\
& \quad\quad\quad\ \mid [v_1, \ldots, v_n] & & \texttt{(LstV)} \\
\text{Environments} & \rho \in \mathbb{X} \rightharpoonup \mathbb{V} & & \texttt{(Env)} \\
\text{Continuations} & \mathbb{K} \ni \kappa ::= \square & & \texttt{(EmptyK)} \\
& \quad\quad\quad\ \mid (\rho \vdash e) :: \kappa & & \texttt{(EvalK)} \\
& \quad\quad\quad\ \mid (+) :: \kappa & & \texttt{(AddK)} \\
& \quad\quad\quad\ \mid (\times) :: \kappa & & \texttt{(MulK)} \\
& \quad\quad\quad\ \mid (@) :: \kappa & & \texttt{(AppK)} \\
& \quad\quad\quad\ \mid (::) :: \kappa & & \texttt{(ConsK)} \\
& \quad\quad\quad\ \mid ([n]) :: \kappa & & \texttt{(LstK)} \\
& \quad\quad\quad\ \mid (\rho \vdash \rhd(e \mid x_1, x_2, e)) :: \kappa & & \texttt{(MatK)} \\
\text{Value Stacks} & \mathbb{S} \ni s ::= \blacksquare \mid v :: s & & \texttt{(List[Value])}
\end{array}
$$

The transition of the abstract machine is defined as follows:

$$\boxed{\langle \kappa \mid\mid s \rangle \longrightarrow \langle \kappa \mid\mid s \rangle}$$

$$
\begin{array}{lll}
\texttt{Num} & \langle (\rho \vdash n) :: \kappa \mid\mid s \rangle & \longrightarrow \langle \kappa \mid\mid n :: s \rangle \\
\\
\texttt{Add}_1 & \langle (\rho \vdash e_1 + e_2) :: \kappa \mid\mid s \rangle & \longrightarrow \langle (\rho \vdash e_1) :: (\rho \vdash e_2) :: (+) :: \kappa \mid\mid s \rangle \\
\texttt{Add}_2 & \langle (+) :: \kappa \mid\mid n_2 :: n_1 :: s \rangle & \longrightarrow \langle \kappa \mid\mid (n_1 + n_2) :: s \rangle \\
\\
\texttt{Mul}_1 & \langle (\rho \vdash e_1 * e_2) :: \kappa \mid\mid s \rangle & \longrightarrow \langle (\rho \vdash e_1) :: (\rho \vdash e_2) :: (\times) :: \kappa \mid\mid s \rangle \\
\texttt{Mul}_2 & \langle (\times) :: \kappa \mid\mid n_2 :: n_1 :: s \rangle & \longrightarrow \langle \kappa \mid\mid (n_1 \times n_2) :: s \rangle \\
\\
\texttt{Id} & \langle (\rho \vdash x) :: \kappa \mid\mid s \rangle & \longrightarrow \langle \kappa \mid\mid \rho(x) :: s \rangle \\
\\
\texttt{Fun} & \langle (\rho \vdash \lambda x.e) :: \kappa \mid\mid s \rangle & \longrightarrow \langle \kappa \mid\mid \langle \lambda x.e, \rho \rangle :: s \rangle \\
\\
\texttt{Def} & \langle (\rho \vdash \texttt{def}\ x(y) = e_1;\ e_2) :: \kappa \mid\mid s \rangle & \longrightarrow \langle (\rho' \vdash e_2) :: \kappa \mid\mid s \rangle \\
 & & \quad \text{where } \rho' = \rho[x \mapsto \langle \lambda y.e_1, \rho' \rangle] \\
\\
\texttt{App}_1 & \langle (\rho \vdash e_1(e_2)) :: \kappa \mid\mid s \rangle & \longrightarrow \langle (\rho \vdash e_1) :: (\rho \vdash e_2) :: (@) :: \kappa \mid\mid s \rangle \\
\texttt{App}_2 & \langle (@) :: \kappa \mid\mid v_2 :: \langle \lambda x.e, \rho \rangle :: s \rangle & \longrightarrow \langle (\rho[x \mapsto v_2] \vdash e) :: \kappa \mid\mid s \rangle \\
\texttt{Cons}_1 & \langle (\rho \vdash e_1 :: e_2) :: \kappa \mid\mid s \rangle & \longrightarrow \langle (\rho \vdash e_1) :: (\rho \vdash e_2) :: (::) :: \kappa \mid\mid s \rangle \\
\texttt{Cons}_2 & \langle (::) :: \kappa \mid\mid v_2 :: v_1 :: s \rangle & \longrightarrow \langle \kappa \mid\mid (v_1 :: v_2) :: s \rangle \\
\\
\texttt{Lst}_1 & \langle (\rho \vdash [e_1, \ldots, e_n]) :: \kappa \mid\mid s \rangle & \longrightarrow \langle (\rho \vdash e_1) :: \ldots :: (\rho \vdash e_n) :: ([n]) :: \kappa \mid\mid s \rangle \\
\texttt{Lst}_2 & \langle ([n]) :: \kappa \mid\mid v_n :: \ldots :: v_1 :: s \rangle & \longrightarrow \langle \kappa \mid\mid [v_1, \ldots, v_n] :: s \rangle \\
\\
\texttt{Mat}_1 & \langle (\rho \vdash e_1 \rhd (e_2 \mid x_1, x_2, e_3)) :: \kappa \mid\mid s \rangle & \longrightarrow \langle (\rho \vdash e_1) :: (\rho \vdash \rhd(e_2 \mid x_1, x_2, e_3)) :: \kappa \mid\mid s \rangle \\
\texttt{Mat}_2 & \langle (\rho \vdash \rhd(e_2 \mid x_1, x_2, e_3)) :: \kappa \mid\mid [] :: s \rangle & \longrightarrow \langle (\rho \vdash e_2) :: \kappa \mid\mid s \rangle \\
\texttt{Mat}_3 & \langle (\rho \vdash \rhd(e_2 \mid x_1, x_2, e_3)) :: \kappa \mid\mid [v_1, v_2, \ldots, v_n]) :: s \rangle & \longrightarrow \langle (\rho[x_1 \mapsto v_1, x_2 \mapsto [v_2, \ldots, v_n]] \vdash e_3) :: \kappa \mid\mid s \rangle
\end{array}
$$

where $\longrightarrow^*$ is the reflexive-transitive closure of $\longrightarrow$ and denotes the repeated transition of the abstract machine.

$$\langle \kappa \mid\mid s \rangle \longrightarrow^* \langle \kappa \mid\mid s \rangle$$

$$\frac{\langle \kappa \mid\mid s \rangle \longrightarrow^* \langle \kappa' \mid\mid s' \rangle \qquad \langle \kappa' \mid\mid s' \rangle \longrightarrow \langle \kappa'' \mid\mid s'' \rangle}{\langle \kappa \mid\mid s \rangle \longrightarrow^* \langle \kappa'' \mid\mid s'' \rangle}$$

The evaluation result of an expression e is the value v if

$$\langle (\varnothing \vdash e) :: \square \mid\mid \blacksquare \rangle \longrightarrow^* \langle \square \mid\mid v :: \blacksquare \rangle$$